

7.1. Коллекции, структуры данных и хранение групп объектов

7.1 Введение в структуры данных

7.1.1 Что такое структура данных?

7.1.2 Зачем вообще нужны структуры данных в программировании?

7.1.3 Чем структура данных отличается от просто набора переменных?

7.1.4 Почему данные часто нужно хранить не по одному значению, а группами?

7.1.5 Какие базовые операции обычно есть у структуры данных?

7.1.6 Почему разные структуры данных подходят под разные задачи?

7.1.7 Что такое доступ, поиск, вставка и удаление?

7.1.8 Почему время выполнения операций зависит от выбранной структуры данных?

7.2 Массивы как первая структура данных

7.2.1 Что такое массив в Java?

7.2.2 Зачем нужен массив и когда его удобно использовать?

7.2.3 Как создать массив в Java?

7.2.4 Почему все элементы массива имеют один и тот же тип?

7.2.5 Что такое индекс массива?

7.2.6 Почему индексация начинается с нуля?

7.2.7 Как получить элемент массива и изменить его значение?

7.2.8 Как узнать длину массива?

7.2.9 Почему размер массива нельзя изменить после создания?

7.2.10 Какие значения по умолчанию получают элементы массива?

7.2.11 Чем массив примитивов отличается от массива объектов?

7.2.12 Что происходит при выходе за границы массива?

7.3 От массивов к коллекциям

- 7.3.1 Какие проблемы у массивов возникают в реальных программах?
- 7.3.2 Почему массив неудобен для частого добавления и удаления элементов?
- 7.3.3 Почему программистам понадобились коллекции?
- 7.3.4 Что значит, что коллекции являются абстракцией над хранением данных?
- 7.3.5 Почему коллекции не магия, а обычные классы, написанные людьми?
- 7.3.6 Когда массив все еще лучше коллекции?

7.4 Общее понимание коллекций в Java

- 7.4.1 Что такое Collections Framework в Java?
- 7.4.2 Какие основные типы коллекций есть в Java?
- 7.4.3 Чем отличается Collection от Collections?
- 7.4.4 Какие базовые операции есть у большинства коллекций?
- 7.4.5 Как понять, какую коллекцию выбрать под задачу?
- 7.4.6 Почему одна и та же операция в разных коллекциях работает по-разному?

7.5 Дженерики и типобезопасность коллекций

- 7.5.1 Зачем коллекциям нужны дженерики?
- 7.5.2 Что дает запись вроде `List<String>`?
- 7.5.3 Что такое типобезопасность коллекций?
- 7.5.4 Почему без дженериков работать с коллекциями опаснее и менее удобно?
- 7.5.5 Что такое raw type и почему его лучше избегать?

7.6 Интерфейс List

- 7.6.1 Что такое List и для каких задач он нужен?
- 7.6.2 Чем List отличается от массива?
- 7.6.3 Почему List хранит элементы в определенном порядке?
- 7.6.4 Почему в List можно хранить дубликаты?
- 7.6.5 Какие основные операции есть у List?
- 7.6.6 Когда List особенно удобен?

7.7 ArrayList

- 7.7.1 Что такое ArrayList и на какой идее он построен?
- 7.7.2 Как ArrayList хранит элементы внутри?
- 7.7.3 Что происходит внутри при добавлении элемента в ArrayList?
- 7.7.4 Что происходит, когда внутренний массив ArrayList заполняется?
- 7.7.5 Чем size отличается от capacity?
- 7.7.6 Как работает get в ArrayList?
- 7.7.7 Как работает add по индексу в ArrayList?
- 7.7.8 Как работает remove в ArrayList?
- 7.7.9 Почему вставка и удаление в середину ArrayList могут быть дорогими?
- 7.7.10 Когда ArrayList особенно хорош, а когда неудобен?

7.8 LinkedList

- 7.8.1 Что такое LinkedList?
- 7.8.2 Чем связный список отличается от массива по идее хранения?
- 7.8.3 Что хранит узел LinkedList?
- 7.8.4 Как LinkedList хранит первый и последний элемент?
- 7.8.5 Как работает добавление в начало и в конец LinkedList?
- 7.8.6 Как работает доступ по индексу в LinkedList?
- 7.8.7 Почему LinkedList плохо подходит для частого случайного доступа?
- 7.8.8 В каких задачах LinkedList может быть полезен?

7.9 Сравнение ArrayList и LinkedList

- 7.9.1 Чем ArrayList и LinkedList отличаются по внутреннему устройству?
- 7.9.2 В каких операциях ArrayList обычно выигрывает у LinkedList?
- 7.9.3 В каких операциях LinkedList может быть удобнее?
- 7.9.4 Почему асимптотика не всегда полностью отражает реальную производительность?

7.10 Интерфейс Set

- 7.10.1 Что такое Set?
- 7.10.2 Чем Set отличается от List?
- 7.10.3 Почему Set не допускает дубликаты?
- 7.10.4 Как Set понимает, что элемент уже есть?
- 7.10.5 Когда Set подходит лучше List?

7.11 equals, hashCode, Comparable и Comparator в коллекциях

- 7.11.1 Что делает equals() при работе с коллекциями?
- 7.11.2 Что делает hashCode() при работе с коллекциями?
- 7.11.3 Почему для коллекций важно не нарушать контракт equals/hashCode?
- 7.11.4 Что такое Comparable?
- 7.11.5 Что такое Comparator?
- 7.11.6 Чем Comparable отличается от Comparator?
- 7.11.7 Почему compareTo и equals не должны противоречить друг другу?

7.12 HashSet

- 7.12.1 Что такое HashSet и на какой структуре он основан?
- 7.12.2 Почему HashSet обычно работает быстро?
- 7.12.3 Как HashSet определяет, есть ли элемент в наборе?
- 7.12.4 Почему для HashSet так важны equals и hashCode?
- 7.12.5 Что происходит при добавлении элемента в HashSet?
- 7.12.6 Что такое коллизия в хеш-структурах?
- 7.12.7 Почему неправильный equals/hashCode ломает работу HashSet?
- 7.12.8 Можно ли менять объект после добавления в HashSet?
- 7.12.9 Чем в общих чертах HashSet отличается от LinkedHashSet и TreeSet?

7.13 Queue, Deque и PriorityQueue

- 7.13.1 Что такое очередь как структура данных?
- 7.13.2 В чем идея FIFO?
- 7.13.3 Что такое Deque?
- 7.13.4 Чем очередь, дек и стек отличаются по поведению?
- 7.13.5 Что такое очередь с приоритетом?
- 7.13.6 В каких задачах Queue, Deque и PriorityQueue особенно полезны?

7.14 Интерфейс Map

- 7.14.1 Что такое Map?
- 7.14.2 Чем Map отличается от Collection?
- 7.14.3 Что такое ключ и значение?
- 7.14.4 Почему ключи в Map должны быть уникальны?
- 7.14.5 Какие основные операции есть у Map?
- 7.14.6 Где Map особенно полезна в реальном коде?

7.15 HashMap

- 7.15.1 Что такое HashMap?
- 7.15.2 Что хранится внутри HashMap на базовом уровне?
- 7.15.3 Что такое бакет в HashMap?
- 7.15.4 Что такое узел записи в HashMap?
- 7.15.5 Как HashMap вычисляет место для хранения ключа?
- 7.15.6 Какую роль играют hashCode и equals в HashMap?
- 7.15.7 Как работает put в HashMap?
- 7.15.8 Как работает get в HashMap?
- 7.15.9 Что происходит при коллизии в HashMap?
- 7.15.10 Что такое load factor и зачем он нужен?
- 7.15.11 Что такое resize и когда он происходит?
- 7.15.12 Почему HashMap перераспределяет элементы при расширении?
- 7.15.13 Почему в современных реализациях HashMap бакет может превращаться в дерево?
- 7.15.14 Почему плохой hashCode ухудшает производительность Map?
- 7.15.15 Почему mutable-ключи опасны для HashMap?
- 7.15.16 Можно ли использовать null как ключ и как значение в HashMap?
- 7.15.17 Почему порядок элементов в HashMap не гарантирован?
- 7.15.18 Чем в общих чертах HashMap отличается от LinkedHashMap и TreeMap?

7.16 Итерация по коллекциям

- 7.16.1 Как пройти по коллекции в Java?
- 7.16.2 Что такое Iterator?
- 7.16.3 Зачем нужен Iterator, если есть foreach?
- 7.16.4 Как безопасно удалять элементы во время обхода коллекции?
- 7.16.5 Почему нельзя просто удалять элементы из коллекции в цикле как попало?
- 7.16.6 Что такое fail-fast поведение итераторов?
- 7.16.7 Что такое ListIterator и чем он отличается от обычного Iterator?

7.17 Arrays, Collections и неизменяемые коллекции

- 7.17.1 Что делает класс Arrays?
- 7.17.2 Что делает класс Collections?
- 7.17.3 Что такое неизменяемая коллекция?
- 7.17.4 Чем неизменяемая коллекция отличается от изменяемой?
- 7.17.5 Зачем нужны unmodifiable-обертки?
- 7.17.6 Почему неизменяемые коллекции полезны для надежности кода?
- 7.18 Сложность операций и выбор коллекции
 - 7.18.1 Что такое временная сложность операций?
 - 7.18.2 Почему при выборе коллекции важно понимать ее внутреннюю реализацию?
 - 7.18.3 Какие операции важнее всего учитывать при выборе структуры данных?
 - 7.18.4 Почему нет одной лучшей коллекции на все случаи жизни?
 - 7.18.5 Как выбирать между List, Set, Queue и Map?
 - 7.18.6 Как выбирать между ArrayList, HashSet, HashMap и другими популярными реализациями?

8. Исключения в Java

- 8.1 Что такое исключение в Java?
- 8.2 Зачем в программе вообще нужен механизм исключений?
- 8.3 Чем Exception отличается от Error?
- 8.4 Что такое checked и unchecked исключения?
- 8.5 Чем RuntimeException отличается от остальных исключений?
- 8.6 Какие исключения чаще всего встречаются у новичков?
- 8.7 Почему возникает NullPointerException?
- 8.8 Что делают try, catch и finally?
- 8.9 Чем throw отличается от throws?
- 8.10 Что значит, что исключение всплывает по стеку вызовов?
- 8.11 Что такое stack trace и как его читать?
- 8.12 Когда исключение нужно обработать, а когда лучше пробросить выше?
- 8.13 Зачем создавать собственные исключения?
- 8.14 Почему важно не терять исходную причину ошибки при оборачивании исключений?
- 8.15 Какие плохие практики чаще всего встречаются при работе с исключениями?
- 8.16 Понятие ресурс, try with resources, как с этим работать